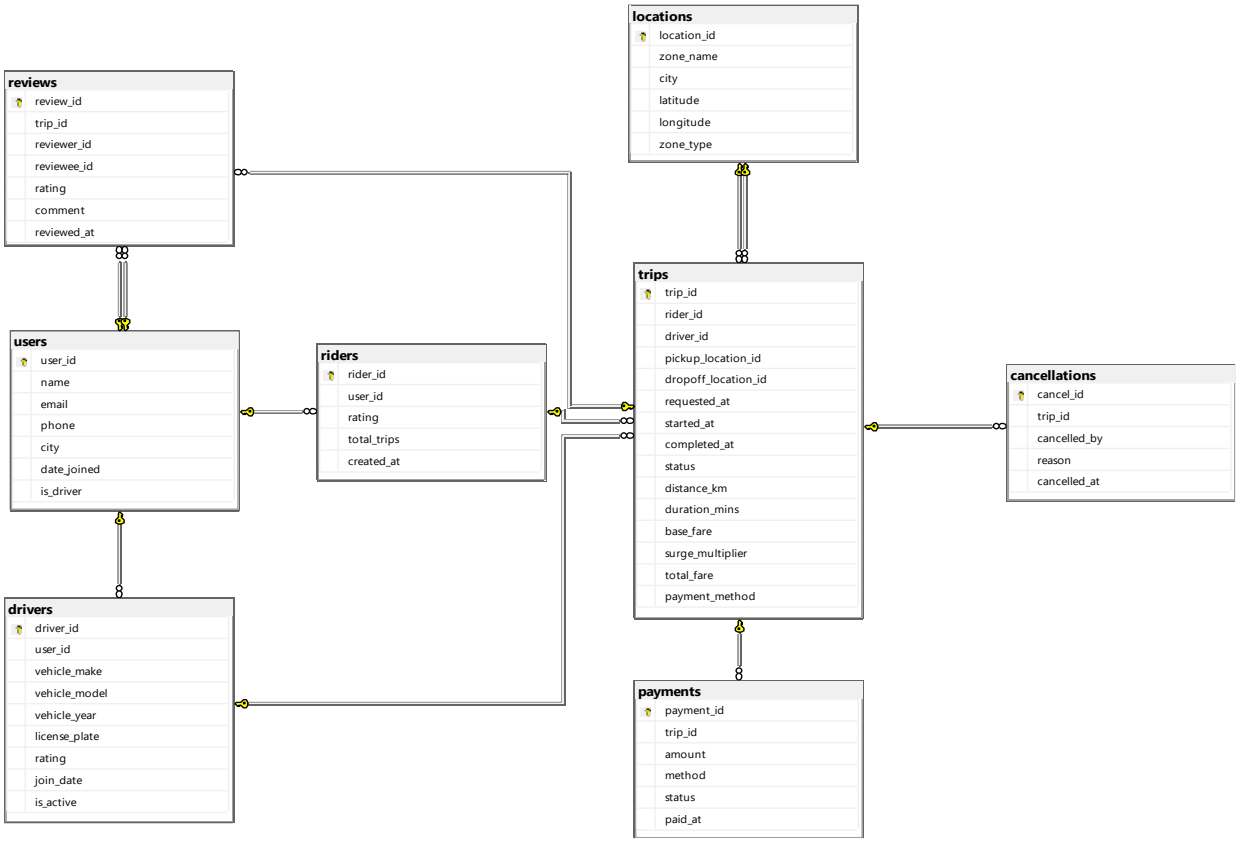
UBER SQL Veritabanı Üzerinde Sorgulamalar



SORGULAR

1. **Sistemdeki tüm kullanıcıların adlarını ve e-posta adreslerini listeleyin.**

```sql
SELECT name, email FROM users;
```

2. **Sadece sürücü olan kullanıcıların listesini getirin.**

```sql
SELECT * FROM users WHERE is_driver = 1;
```

3. **New York şehrinde yaşayan kullanıcıları bulun.**

```sql
SELECT * FROM users WHERE city = 'New York';
```

4. **Puanı (rating) 4.5'in üzerinde olan tüm sürücüleri listeleyin.**

```sql
SELECT * FROM drivers WHERE rating > 4.5;
```

5. **Aktif olmayan (is_active = 0) sürücülerin araç marka ve modellerini listeleyin.**

```sql
SELECT vehicle_make, vehicle_model FROM drivers WHERE is_active = 0;
```

6. **Sistemde kayıtlı kaç farklı lokasyon (zone_name) olduğunu bulun.**

```sql
SELECT COUNT(DISTINCT zone_name) FROM locations;
```

7. **'Airport' tipindeki lokasyonları listeleyin.**

```sql
SELECT * FROM locations WHERE zone_type = 'Airport';
```

8. **Tamamlanmış (status = 'completed') tüm yolculukların detaylarını getirin.**

```sql
SELECT * FROM trips WHERE status = 'Completed';
```

9. **Ödeme yöntemi 'cash' olan yolculukları listeleyin.**

```sql
SELECT * FROM payments WHERE method = 'Cash';
```

10. **İptal edilen yolculukların iptal nedenlerini ve kim tarafından iptal edildiğini listeleyin.**

```sql
SELECT cancelled_by, reason FROM cancellations;
```

11. **Her şehirde kaç kullanıcı olduğunu listeleyin.**

```sql
SELECT city, COUNT(*) FROM users GROUP BY city;
```

12. **Sürücülerin isimlerini ve kullandıkları araçların plaka numaralarını birlikte getirin.**

```sql
SELECT drivers.license_plate, users.name  FROM drivers  JOIN users ON
users.user_id = drivers.user_id
```

13. **Her bir sürücü toplamda kaç yolculuk tamamlamıştır?**

```sql
SELECT driver_id, COUNT(*) as completed_trips
FROM trips
WHERE status = 'completed'
GROUP BY driver_id;
```

14. **En çok yolculuk yapan ilk 5 yolcuyu (rider) ve toplam yolculuk sayılarını bulun.**

```sql
SELECT TOP 5 users.name, COUNT(trips.trip_id) as toplam_yolculuk
FROM  users JOIN riders ON users.user_id = riders.user_id
          JOIN trips ON trips.rider_id = riders.rider_id
GROUP BY users.name, riders.rider_id
ORDER BY toplam_yolculuk DESC
```

15. **Her bir şehir bazında ortalama sürücü puanlarını hesaplayın.**

```sql
SELECT users.city, ROUND(AVG(drivers.rating),2) as ortalama_puan
FROM users JOIN drivers ON users.user_id = drivers.user_id
GROUP BY users.city
```

16. **Hangi araç markası sistemde en çok kullanılmaktadır?**

```sql
SELECT TOP 1 drivers.vehicle_make, COUNT(*) arac_sayisi
FROM drivers
GROUP BY drivers.vehicle_make
```

17. **Her bir yolculuk için; yolcu adını, sürücü adını ve yolculuk tutarını listeleyin.**

```sql
SELECT ur.name as rider_name, ud.name as driver_name, t.total_fare
FROM trips t
JOIN riders r ON t.rider_id = r.rider_id JOIN users ur ON r.user_id =
ur.user_id
JOIN drivers d ON t.driver_id = d.driver_id JOIN users ud ON d.user_id =
ud.user_id;
```

18. **Başlangıç noktası havalimanı ('airport') olan yolculukların toplam sayısını bulun.**

```sql
SELECT COUNT(*) FROM trips t
JOIN locations l ON t.pickup_location_id = l.location_id
WHERE l.zone_type = 'airport';
```

19. **Ödeme durumu 'success' olan ödemelerin toplam tutarını hesaplayın.**

```sql
SELECT SUM(amount) FROM payments WHERE status = 'success';
```

20. **Sürücülerin araç model yıllarına göre dağılımını (kaç adet 2020 model araç var vb.) bulun.**

```sql
SELECT vehicle_year, COUNT(*) FROM  drivers GROUP BY vehicle_year;
```

21. **2023 yılında sisteme katılan kullanıcıların listesini getirin.**

```sql
SELECT name FROM  users WHERE date_joined
BETWEEN '2023-01-01' AND '2023-12-31';
```

22. **Yolculuk başına ortalama kat edilen mesafe (distance_km) nedir?**

```
SELECT ROUND(AVG(distance_km),2) FROM trips;
```

23. **En yüksek toplam kazanca sahip ilk 3 sürücüyü ve kazandıkları tutarı listeleyin.**

```
SELECT TOP 3 driver_id, SUM(total_fare) as toplam_kazanc FROM trips
GROUP BY driver_id ORDER BY toplam_kazanc DESC;
```

24. **Hangi bölgeler (zone_name) en sık 'pickup' (başlangıç) noktası olarak kullanılmaktadır?**

```
SELECT l.zone_name, COUNT(*) as pickup_count
FROM trips t JOIN locations l ON t.pickup_location_id = l.location_id
GROUP BY l.zone_name ORDER BY pickup_count DESC;
```

25. **Sürücülerin kendi şehirleri dışındaki şehirlerde yaptıkları yolculukları listeleyin.**

```
SELECT u.name, u.city as home_city, l.city as trip_city
FROM trips t
JOIN drivers d ON t.driver_id = d.driver_id JOIN users u ON d.user_id =
u.user_id
JOIN locations l ON t.pickup_location_id = l.location_id
WHERE u.city <> l.city;
```

26. **Bir yolculuğun talep edilmesi (requested_at) ile başlaması (started_at) arasındaki ortalama bekleme süresini hesaplayın.**

```
SELECT AVG(DATEDIFF(MINUTE, started_at, requested_at))
FROM trips
WHERE started_at IS NOT NULL
```

27. **Hiç yolculuk yapmamış olan kayıtlı yolcuların isimlerini bulun.**

```
SELECT u.name
FROM users u JOIN riders r ON u.user_id = r.user_id
LEFT JOIN trips t ON r.rider_id = t.rider_id
WHERE t.trip_id IS NULL
```

28. **Her ay için toplam yolculuk sayısını listeleyerek büyüme trendini analiz edin.**

```
SELECT MONTH(requested_at) as aylar, COUNT(*) as toplam_yolculuk_sayisi
FROM trips
GROUP BY MONTH(requested_at)
ORDER BY aylar
```

29. **'Surge multiplier' (fiyat artış katsayısı) uygulanan yolculukların tüm yolculuklara oranını bulun.**

```
SELECT (COUNT(CASE WHEN surge_multiplier > 1 THEN 1 END) *100 / COUNT(*))
as artis_yuzdesi
FROM trips
```

30. **Aynı gün içinde 5'ten fazla yolculuk yapan sürücüleri listeleyin.**

```
SELECT
    driver_id,
    CAST(requested_at AS DATE) AS trip_date,
    COUNT(*) AS trip_count
FROM trips
GROUP BY
    driver_id,
    CAST(requested_at AS DATE)
HAVING COUNT(*) > 2;
```

31. **Ödeme yöntemi bazında (cash, wallet, card) toplam ciro dağılımını hesaplayın.**

```
SELECT method, ROUND(SUM(amount),2) as toplam_ciro
FROM payments
GROUP BY method
```

32. **İptal edilen yolculukların en yaygın 3 iptal sebebini bulun.**

```sql
SELECT TOP 3 reason, COUNT(*) as toplam
FROM cancellations
GROUP BY reason
ORDER BY toplam DESC
```

33. **Hem sürücü hem de yolcu olarak sistemde kayıtlı olan "hibrit" kullanıcıları listeleyin.**

```sql
SELECT name
FROM users
WHERE user_id IN (SELECT user_id FROM drivers)
AND user_id IN (SELECT user_id FROM riders);
```

34. **Yolculuk süresi (duration_mins) 60 dakikadan fazla olan yolculukları ve bu yolculukların sürücülerini getirin.**

```sql
SELECT
    u.name AS driver_name,
    t.trip_id,
    t.duration_mins
FROM trips t
JOIN drivers d ON t.driver_id = d.driver_id
JOIN users u ON d.user_id = u.user_id
WHERE t.duration_mins > 60;
```

35. **Hafta sonları yapılan toplam yolculuk sayısı ile hafta içi yapılanları karşılaştırın.**

```sql
SELECT
CASE WHEN DATEPART(WEEKDAY, requested_at) IN (1, 7) THEN 'hafta sonu'
ELSE 'hafta içi'
END as gun_turu,
COUNT(*) as toplam_yolculuk
FROM trips
GROUP BY
CASE WHEN DATEPART(WEEKDAY, requested_at) IN (1, 7) THEN 'hafta sonu'
ELSE 'hafta içi'
END
```

36. **Her bir bölge tipi (commercial, residential, airport) için ortalama total_fare miktarını bulun.**

```sql
SELECT l.zone_name, ROUND(AVG(t.total_fare),2) as ort_toplam_ucret
FROM locations l JOIN trips t ON l.location_id = t.pickup_location_id
GROUP BY l.zone_name
```

37. **En düşük puana (rating) sahip 10 sürücüyü ve aldıkları toplam yorum sayısını listeleyin.**

```sql
SELECT TOP 10 u.name, d.rating
FROM drivers d JOIN users u ON d.user_id = u.user_id
ORDER BY d.rating;
```

38. **Sürücülerin sisteme katılım tarihinden itibaren geçen süreye göre ortalama puanlarını karşılaştırın (Kıdemli vs. Yeni sürücüler).**

```sql
SELECT CASE
WHEN join_date < '2023-01-01' THEN 'Kıdemli' ELSE 'Yeni Sürücü' END as Kıdem,
AVG(rating)
FROM drivers
GROUP BY
CASE WHEN join_date < '2023-01-01' THEN 'Kıdemli' ELSE 'Yeni Sürücü' END
```

39. **Gece saatlerinde (22:00 - 05:00) yapılan yolculukların toplam gelirini hesaplayın.**

```sql
SELECT SUM(total_fare) as toplam_gelir
FROM trips
WHERE DATEPART(HOUR, requested_at) >= 22 OR DATEPART(HOUR, requested_at) < 5
```